

REPORTE DE PRÁCTICA 01: BATALLA NAVAL (TCP / UDP)

Experiencia Educativa: Programación en Red

Alumno: Jorge Gael Hernández Marcial

Fecha: 09/09/2026



1. Objetivo de la Práctica

Diseñar e implementar una arquitectura de comunicación básica bajo el modelo Cliente-Servidor utilizando el lenguaje de programación Python y la librería estándar socket, con el fin de comprender el funcionamiento del protocolo de transporte TCP en la transmisión confiable y ordenada de datos orientados a conexión.

2. Marco Teórico / Fundamentos de Red

- **Modelo Cliente-Servidor:** Arquitectura de red donde un programa (el servidor) espera y gestiona las solicitudes de recursos o datos provenientes de uno o más programas clientes.
- **Sockets:** Un socket es un punto de conexión final (endpoint) de comunicación bidireccional a través de una red informática. Permite que procesos distintos intercambien información tanto a nivel local como en redes remotas.
- **Protocolo TCP (Transmission Control Protocol):** Protocolo orientado a conexión que garantiza que los paquetes de datos lleguen en el orden correcto, sin duplicados y sin pérdida, estableciendo un enlace firme (Handshake) antes de realizar la transferencia de información.

3. Desarrollo y Metodología

A. Implementación del Servidor (servidor.py)

El programa del servidor opera bajo los siguientes pasos lógicos en la red:

1. **Creación del socket:** Se instancia un socket utilizando direcciones IPv4 (AF_INET) y el protocolo de flujo TCP (SOCK_STREAM).
2. **Asociación (bind):** Se vincula el socket a una dirección IP local (127.0.0.1) y a un puerto específico (65432) para habilitar el punto de escucha del servicio.
3. **Puesta en escucha (listen):** Se configura el servidor para aceptar solicitudes entrantes de los clientes en cola.
4. **Aceptación de conexiones (accept):** El servidor se bloquea temporalmente a la espera de un cliente; al establecerse la conexión, se genera un canal dedicado y se extrae la información de la IP remota.
5. **Recepción y respuesta (recv / sendall):** A través de un bucle, el servidor recibe los bytes enviados por el cliente (coordenadas de ataque), los decodifica a formato de texto UTF-8 y procesa una respuesta simulada que es devuelta con sendall().

B. Implementación del Cliente (cliente.py)

El script del cliente cumple con la contraparte operativa:

1. **Instanciación y Conexión (connect):** Crea su propio socket TCP y establece una conexión directa con la dirección IP y el puerto definidos por el servidor.
2. **Transmisión de datos (sendall):** Codifica un mensaje de texto (coordenada de ataque, por ejemplo, "B4") a bytes y lo transmite de forma segura a través de la red.
3. **Recepción de confirmación:** Espera la respuesta emitida por el servidor, la procesa y la muestra en la consola local antes de cerrar la sesión de forma limpia con close().
4. **Evidencias de Ejecución (Capturas de Pantalla)**

```
PS C:\Users\Jorge\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Jorge\AppData\Local\Programs\Python\Python38-64\python.exe -c "import sys; sys.path.append('C:/Users/Jorge/Desktop/Actividad Práctica batalla naval TCP _ UDP/servidor.py'); import servidor; servidor.servidor()"
Servidor de Batalla Naval activo y escuchando en 127.0.0.1:65432...
```

```
Conexión establecida con éxito al servidor de Batalla Naval.  
Enviando coordenada de ataque: B4  
Respuesta recibida del servidor: Procesando jugada para la coordenada: B4 -> Resultado: [Agua / Impacto]  
PS C:\Users\Jorge\AppData\Local\Programs\Microsoft VS Code>
```

- Figura 1: Servidor ejecutándose y mostrando el estado de escucha activa en la dirección y puerto configurados.
- Figura 2: Cliente ejecutándose de manera independiente, enviando la coordenada de ataque y recibiendo la respuesta de confirmación del servidor.

5. Conclusiones

La práctica permitió comprobar de forma práctica el comportamiento subyacente de las comunicaciones de red mediante sockets. El uso del protocolo TCP demostró ser fundamental para este tipo de aplicaciones (como un juego de Batalla Naval), ya que asegura que las coordenadas o comandos enviados por el jugador lleguen íntegros y sin alteraciones al servidor. Asimismo, el manejo estructurado de las funciones de red en Python facilitó la visualización clara del ciclo de vida de una conexión, desde la apertura del puerto hasta el cierre seguro de los descriptores de archivo.